Smart Contract Assessment

Moonwell

HALBORN

Summary

0% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

ASSESSMENT TYPE
Assurance Assessment

DATE OF ENGAGEMENT
Jul 9, 2026 - Jul 10, 2026

SEVERITY BREAKDOWN

2	2	0	0
All Findings	Pending	Solved	Risk Accepted

Critical	0	
High	0	
Medium	0	
Low	2	Pending 2
Informational	0	

INTRODUCTION

`Moonwell` engaged `Halborn` to perform a security assessment of their smart contracts on Jul 10th, 2026. The assessment scope was limited to the smart contracts provided to Halborn. Commit hashes and additional details are available in the Scope section of this report.

The `Moonwell` codebase in scope consists of changes to the Wormhole bridge adapter's onchain quoter path, replacing exact payment enforcement with a live quote floor plus surplus refund, and to the OEV price wrappers, which now snapshot and restore the cached round id so the fresh round unlock is scoped to a single nested liquidation rather than persisted globally.

ASSESSMENT SUMMARY

`Halborn` was allocated 1 day for this engagement and assigned 1 full-time security engineer to conduct a comprehensive review of the smart contracts within scope. The engineer is an expert in blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment are to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, `Halborn` identified several areas for improvement to reduce the likelihood and impact of security risks, which should be addressed by the `Moonwell` team. The main recommendations are:

- `Pull the loan token from the caller before advancing cachedRoundId, so no attacker-controlled transfer hook can execute while the fresh price is globally unlocked.`
- `Consider making the surplus refund resilient, for example by allowing the caller to specify a refund recipient, so a caller that cannot receive ETH is not forced to revert or forfeit the surplus.`

TEST APPROACH AND METHODOLODY

`Halborn` conducted a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy within the scope of this assessment. While manual testing is crucial for identifying flaws in logic, processes, and implementation, automated testing enhances coverage of smart contracts and quickly detects deviations from established security best practices.
The following phases and associated tools were employed throughout the term of the assessment:

- Research into the platform's architecture, purpose and use.
- Manual code review and walkthrough of smart contracts to identify any logical issues.
- Comprehensive assessment of the safety and usage of critical Solidity variables and functions within scope that could lead to arithmetic-related vulnerabilities.
- Local testing using custom scripts (`Foundry`).
- Fork testing against main networks (`Foundry`).
- Static security analysis of scoped contracts, and imported functions (`Slither`).

CAVEATS

The current security assessment was focused on evaluating the changes made to the files in scope, as introduced in the following PRs:

- https://github.com/moonwell-fi/moonwell-contracts-v2/pull/665/changes/BASE..614d6959fcb4093dc63504c24a1522983460554a
- https://github.com/moonwell-fi/moonwell-contracts-v2/pull/670/changes/BASE..5b9a817681ed23136d89728eff53dd88155984f2

It's important to note that despite these caveats, the security assessment aimed to provide a thorough evaluation of the protocol's security posture. However, the limitations mentioned above should be considered when interpreting the findings and recommendations.

Risk Methodology

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

5.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A)	1
	Specific (AO:S)	0.2
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

5.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

5.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
	Unchanged (S:U)	1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

Critical	High	Medium	Low	Informational
9 - 10	7 - 8.9	4.5 - 6.9	2 - 4.4	0 - 1.9

Scope

URLS

(a) URL:
https://github.com/moonwell-fi/moonwell-contracts-v2/pull/670/changes/BASE..5b9a817681ed23136d89728eff53dd88155984f2

(b) Items in scope:

- src/xWELL 1 file
  - WormholeBridgeAdapter.sol Solidity

Out-of-Scope: Third party dependencies and economic attacks.

(a) URL:
https://github.com/moonwell-fi/moonwell-contracts-v2/pull/665/changes/BASE..614d6959fcb4093dc63504c24a1522983460554a

(b) Items in scope:

- src/oracles 2 files
  - ChainlinkOEVMorphoWrapper.sol Solidity
  - ChainlinkOEVWrapper.sol Solidity

Out-of-Scope: Third party dependencies and economic attacks.

Out-of-Scope: New features/implementations after the remediation commit IDs.

Assessment Summary & Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
	Overpaid bridge reverts when the caller cannot receive the ETH refund	Low	2.5	Pending
	Free liquidations and OEV fee bypass due to loan-token reentrancy while the fresh price is unlocked	Low	2.5	Pending

Findings & Tech Details

Pending

Overpaid bridge reverts when the caller cannot receive the ETH refund

Low · 2.5 `AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N`

DESCRIPTION

The `WormholeBridgeAdapter._bridgeOut()` onchain-quoter overload refunds any surplus above the live quote with `(bool success, ) = msg.sender.call{value: refund}("")` followed by `require(success, "WormholeBridge: refund failed")`. This is the mechanism that lets a caller over-fund the bridge and get the excess back.

However, the hard `require(success)` means the entire bridge call reverts whenever the refund transfer fails, which happens if `msg.sender` is a contract without a payable `receive`/`fallback` or one whose receipt hook consumes more than the forwarded gas. The intended beneficiary of this feature is a governance execution contract that over-funds with a buffer; if that executing contract cannot accept ETH, every over-funded bridge through the onchain-quoter path reverts, blocking the very use case the upgrade was built for.

RECOMMENDATION

Confirm that every intended caller (in particular the governance execution contract) can receive ETH. Consider making the surplus refund resilient, for example by allowing the caller to specify a refund recipient, so a caller that cannot receive ETH is not forced to revert or forfeit the surplus.

Pending

Free liquidations and OEV fee bypass due to loan-token reentrancy while the fresh price is unlocked

Low · 2.5 `AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N`

DESCRIPTION

The `ChainlinkOEVMorphoWrapper.updatePriceEarlyAndLiquidate()` function is the paid entry point of an OEV price wrapper that serves as the `BASE_FEED_1` for one or more approved Morpho Blue markets. It delays the newest Chainlink price in `latestRoundData()` until a liquidator pays for it, serving the fresh price globally to every Morpho market that uses this wrapper as base feed only while `cachedRoundId` equals the freshest `roundId`. To scope that fresh-price unlock to the paying liquidation, the function snapshots the prior value into `previousCachedRoundId`, advances `cachedRoundId` to the fresh `roundId`, pulls the loan token from the caller with `safeTransferFrom`, calls `morphoBlue.liquidate(...)`, and then restores `cachedRoundId` before the trailing excess-token refund.

However, the restore of `cachedRoundId` is placed after the initial loan-token pull, so `IERC20(loanToken).safeTransferFrom(msg.sender, address(this), maxRepayAmount)` still executes while the fresh price is globally unlocked. The snapshot-and-restore that bounds the unlock protects only the trailing excess-token refund and the collateral fee-split transfers, which run after the restore; the initial pull remains inside the unlocked window. This issue is only exploitable if an approved market uses a callback-enabled loan token (e.g. an ERC777 token, an ERC1363 token, or another token implementing equivalent transfer hooks). In that case, the liquidator receives control during the token transfer. The contract's `nonReentrant` guard only prevents re-entry into `ChainlinkOEVMorphoWrapper` itself; it does not prevent the attacker from calling Morpho Blue's native `liquidate()` directly on other positions. The sibling `ChainlinkOEVWrapper` performs its loan-token pull before advancing `cachedRoundId`, so the pull never runs inside the unlocked window there.

If an approved market is configured with such a callback-enabled loan token, the attacker can liquidate real borrowers in any approved market that shares this wrapper as `BASE_FEED_1` at the fresh, unlocked price using Morpho Blue's native `liquidate()`, capturing the full liquidation incentive and paying zero OEV fee to `feeRecipient`. This breaks the fee-capture invariant the wrapper exists to enforce and grants free liquidations of positions that are only underwater at the fresh price.

For example:

1. Moonwell approves a Morpho market whose loan token calls back the sender on transfer (for example an ERC777 token), with this wrapper as `BASE_FEED_1`.

2. The attacker registers a send hook for that token and identifies real borrowers in approved markets that become liquidatable at the fresh Chainlink price but are still healthy at the delayed price.
3. The attacker calls `updatePriceEarlyAndLiquidate(marketParams, borrower, seizedAssets, maxRepayAmount)` for an approved market that uses that loan token.
4. The function snapshots `cachedRoundId`, advances it to the fresh `roundId` to unlock the fresh price globally, then executes `safeTransferFrom`, invoking the attacker's hook while the price is still unlocked and before the restore runs.
5. Inside the hook the attacker calls Morpho Blue's native `liquidate()` on the real underwater positions; Morpho reads this wrapper's now-fresh price and permits the liquidations, so the attacker seizes collateral at the full incentive with no OEV protocol fee.
6. The hook returns, the wrapper completes its own liquidation, and `cachedRoundId` is restored, hiding the manipulation.

RECOMMENDATION

Pull the loan token from the caller before advancing `cachedRoundId`, so no attacker-controlled transfer hook can execute while the fresh price is globally unlocked. This mirrors the ordering already used by the sibling `ChainlinkOEVWrapper.updatePriceEarlyAndLiquidate()`.

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.